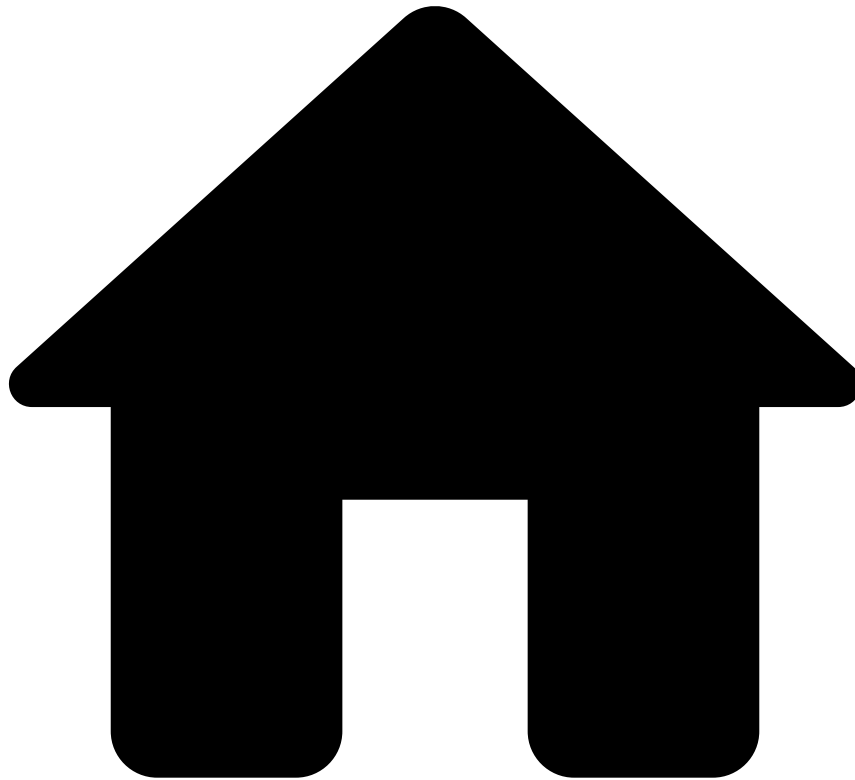


•



- [Core Concepts](#)
- Plans

On this page

Plans

Was this page helpful? 

Data science plans are functions that receive data and produce a transformation of that data (not necessarily with equal size).

Plans do much of the heavy processing required before you train your machine learning [models](#) in the [data science loop](#). Plans are executed as distributed jobs in Spark which allows handling large datasets (e.g. hundreds or even thousands of TBs). In addition, they allow you to natively deploy a subset of these data transformations to a runtime [workflow](#) while keeping the same semantics as in the data science stage.

Check the [operators reference](#) to learn more about the possible elements you can include in your data science plans.

What are plans used for?

Plans define a transformation of one or more [data sources](#) into another data source. They are a vital concept in Pulse and they are a powerful contribution to the data science set of tools. Plans represent a way to boost the performance of:

- **Data scientists:** Delivering a set of tools that make it easier to process data for the training of machine learning models.
- **Engineers:** Abstracting a lot of the usual complexity involved in deploying data science work into production. With a fully integrated modeling and run-time environment, Pulse enables a seamless deployment of data transformations, without any major effort.

In a plan, it is possible to add, update and remove fields, calculate [metrics](#) on the data source, and even join data sources.

Plans are useful to systematize data cleaning, data processing, and feature engineering, both in a data science pipeline and in production. Typically, a data science project will have several plans that are executed in sequence and generate new data sources each as the [output](#). These intermediate data sources are subsequently used as input for other plans.

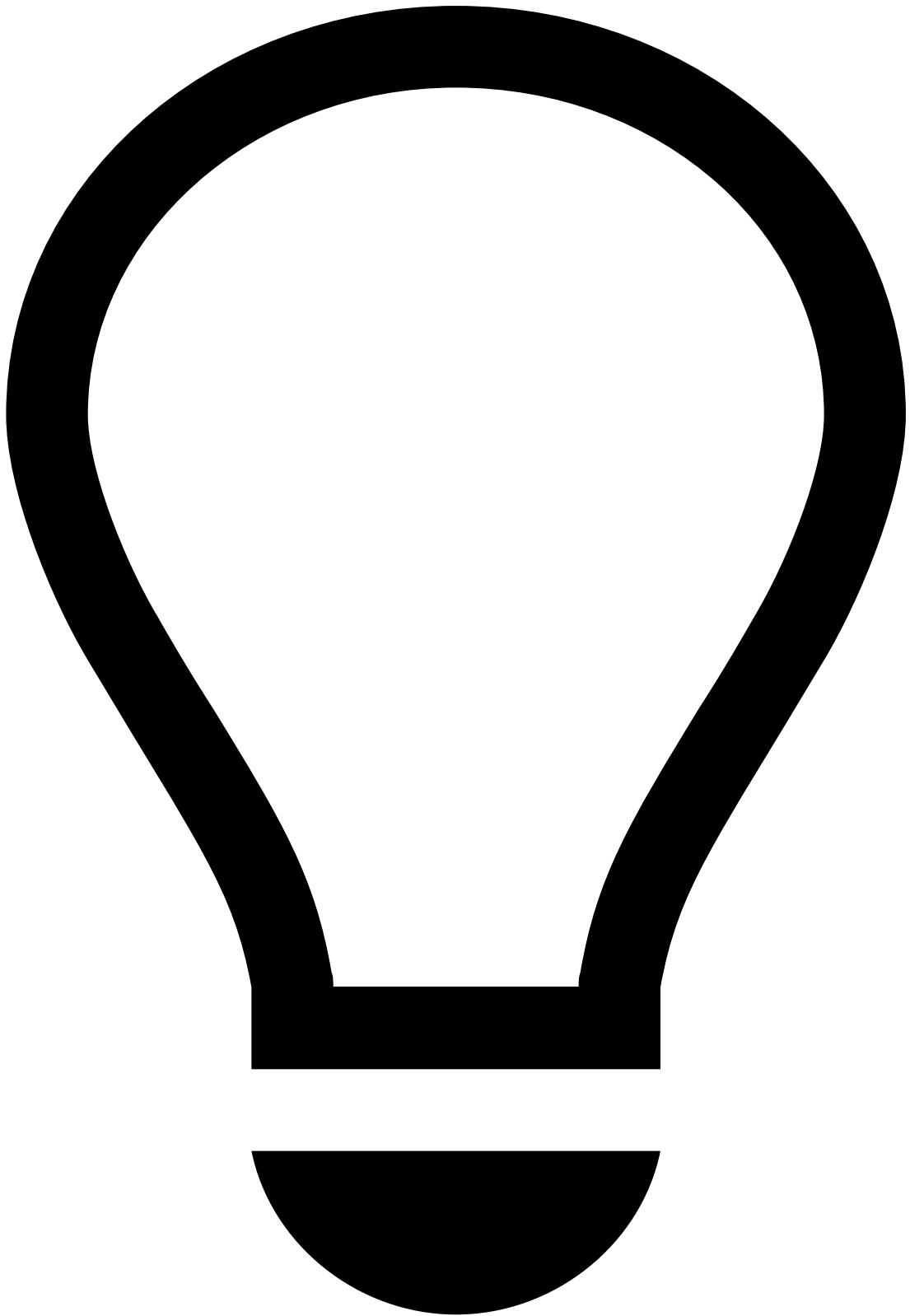
Types of plans

Pulse lets you design three main types of plans, each one aimed at different stages in your project:

- **Deployable plans**
It allows you to design and test, during the data science stage, the data transformations that you will use in a runtime workflow to process real-time events. For example, one or more plans that generate all features that your deployed machine learning [model](#) needs to output a decision in runtime. Pulse ensures that all transformations in a deployable plan will work in runtime exactly as you designed them during the data science loop.
- **Deployable omnichannel plans**
In addition to the functionality offered by regular deployable plans, this type of plan allows to process data from multiple sources, to enable [channels](#). Each input data source corresponds to a separate channel. At runtime, when omnichannel plans are deployed to the workflow, multiple workflows feed into the plan.
- **Non-deployable plans**
The purpose of these plans is to perform data transformations that are purely for offline data exploration purposes, during the data science loop, and that will not be directly deployed in a runtime workflow. For example, a plan that addresses quality issues in your raw data, a plan that samples your data into a more representative dataset, or a plan that joins the raw transaction data with the respective labels, e.g. *fraud/not fraud*.

Deployable plans

A subset of the [transformations available](#) in plans can be deployed into production - typically the plan, or plans, that define the features to be used as input for your machine learning models - so that events can then be turned into instances and scored in real-time.

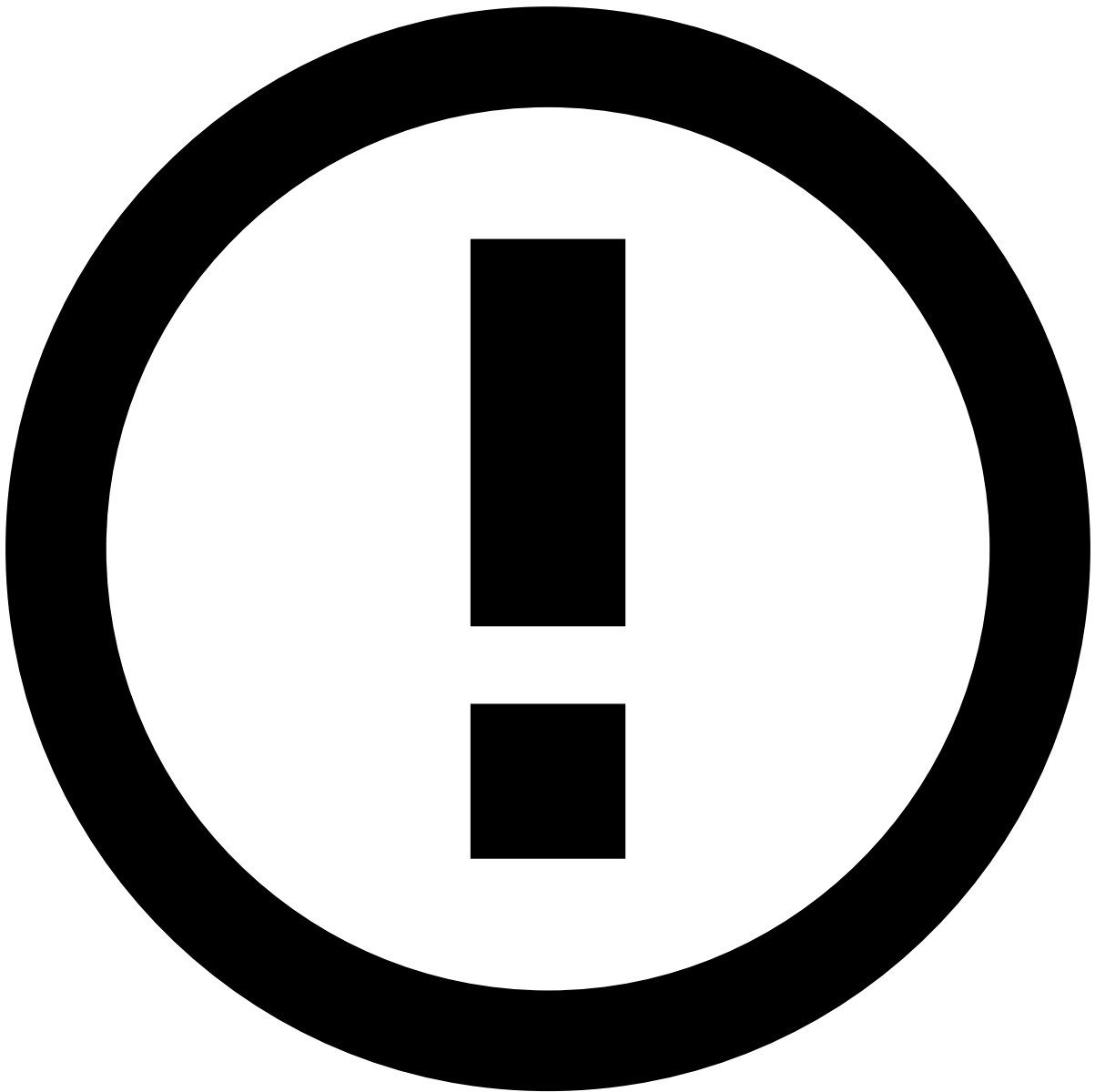


Using Plans in Production

This is extremely important since Pulse guarantees that such subset of the plans functionality will work properly in production for you to be able to score your instances in real-time.

That is very powerful, since in a deployable plan you devise a single piece of logic, and Pulse will apply the same semantics:

- Offline during your data science iterations with great scalability.
- In real-time, in production, with very low latency.



Workflow branching

Due to the fact that Pulse does not support parallel branches in runtime workflows, deployable plans only allow for a single data source to be configured as the input, and only one sink as the only output.

Integrated exploration and runtime stages

A unique feature to Pulse is enabling you to deploy feature plans into runtime without effort. This tight integration of data science and engineering represents a huge boost in performance and productivity.

The way deployable plans work in Pulse removes the obstacles that commonly affect your project when it is ready to go into production. Pulse ensures the following:

- Your data science work is seamlessly deployed into Production without any need for additional development.
- Semantics do not change whether you are exploring your data and modeling your project, or your work is being executed in a production scenario. For example:
 - How Pulse partitions your data in a distributed environment.
 - How Pulse performs real-time and scheduled aggregations.

- Off-the-shelf integration with Cassandra and batch processes to replicate aggregations with long time windows.
- Execution in a high-performance streaming environment delivering sub-millisecond latencies.

The following image depicts a deployable plan — in this case, a plan that filters out internal transactions, unnecessary for the model, and generates metrics and features. Note that the plan's internal definition already includes a single [Source operator](#) and a single [Sink operator](#) (not editable) to ensure that the plan executes a linear path when deployed in runtime.

Deployable omnichannel plans

In addition to the functionality offered by regular deployable plans, deployable omnichannel plans allow to process data from multiple sources, to enable [channels](#).

These plans support working with two or more channels:

- As explained in the [Channels](#) page, there is always a single main channel. When you place the plan in the Workflow, this is the channel processed and scored by that Workflow.
- The remaining channels are secondary channels — they are only fed into the plan for additional information and to improve the detection on the main channel.

The main channel is equivalent to the input source in non-omnichannel plans. You can perform all the same transformations on the main channel that you would do in non-omnichannel plans.

The secondary channels can only be used in aggregations (in the [Metrics operator](#)).

For example, let's say that you are processing ATM withdrawals (debit channel) and would like to check if the customer has recently logged into internet banking with a new device and increased the withdrawal limit (internet banking channel). In this case, *debit* is the main channel (because you are scoring debit card events) and *internet banking* is the secondary channel (because you are consulting it to improve the detection). See the [Creating metrics](#) page for more information on how to implement the relevant aggregation in Pulse.

At runtime, omnichannel plans are fed by workflows:

- The workflow where the plan is deployed corresponds to the main channel, i.e. the data passing through this workflow will be sent to the input of the plan.
- For each secondary channel, you will need to select a different workflow which is processing data for that channel. That data will be automatically fed into the plan.

Non-deployable plans

A non-deployable plan is used only for data exploration purposes and any ETL task, for example in traditional data manipulation when joining and grouping data. When processing a data stream in real-time, in a Production environment, for example, to classify transactions as fraud or not, these activities are not repeatable nor applicable, such as joining data from separate sources.

These plans have no restrictions as to the [available operators](#) they can execute.

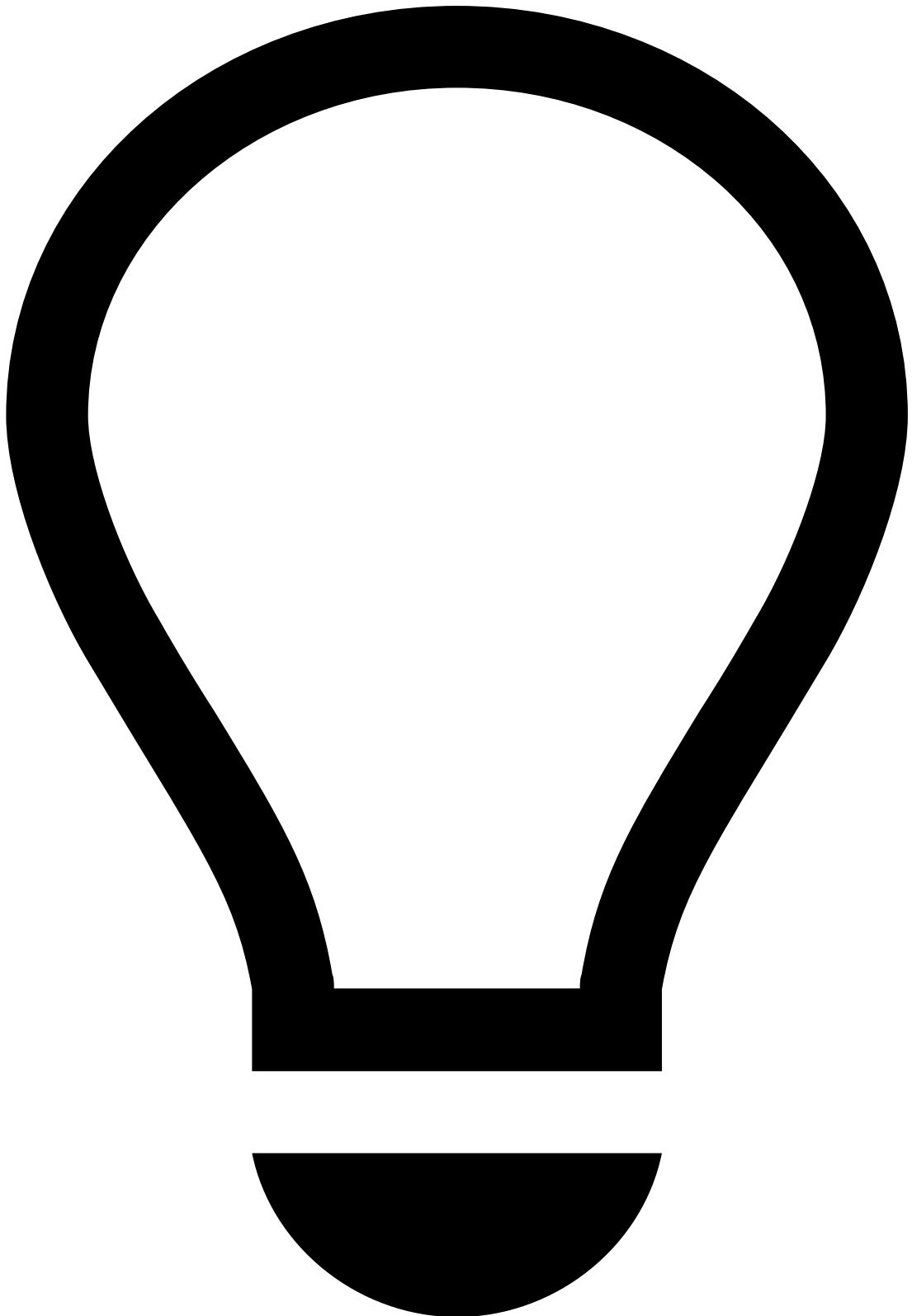
The following image depicts a non-deployable plan that joins data from different sources, in this case, historical transactions with the respective labels, and performs data cleaning operations such as removing unnecessary fields.

Plan execution

A plan is just a definition, which means that there isn't actually any data bound to it, only an input [data schema](#). To actually compute the output data source it is necessary to run it.

Running the plan creates a plan execution. A plan execution represents a snapshot of the plan at the moment it ran, the inputs used and the outputs created, during the data science stage. It allows you to deploy a (deployable) plan to runtime with the same exact configuration and semantics it had when you ran it initially.

For all [operators](#) that require you to define expressions (for example, adding and updating fields with the [Map operator](#)), you must do so with the [Pulse Query Language \(PQL\)](#). Thus, whenever Pulse runs data through a plan, whether it is during the data science stage or in real-time, Pulse is actually executing [PQL commands](#).



Distributed execution

The execution of plans runs distributed in Spark, so Pulse is able to transform data in parallel within a set of machines of a cluster.

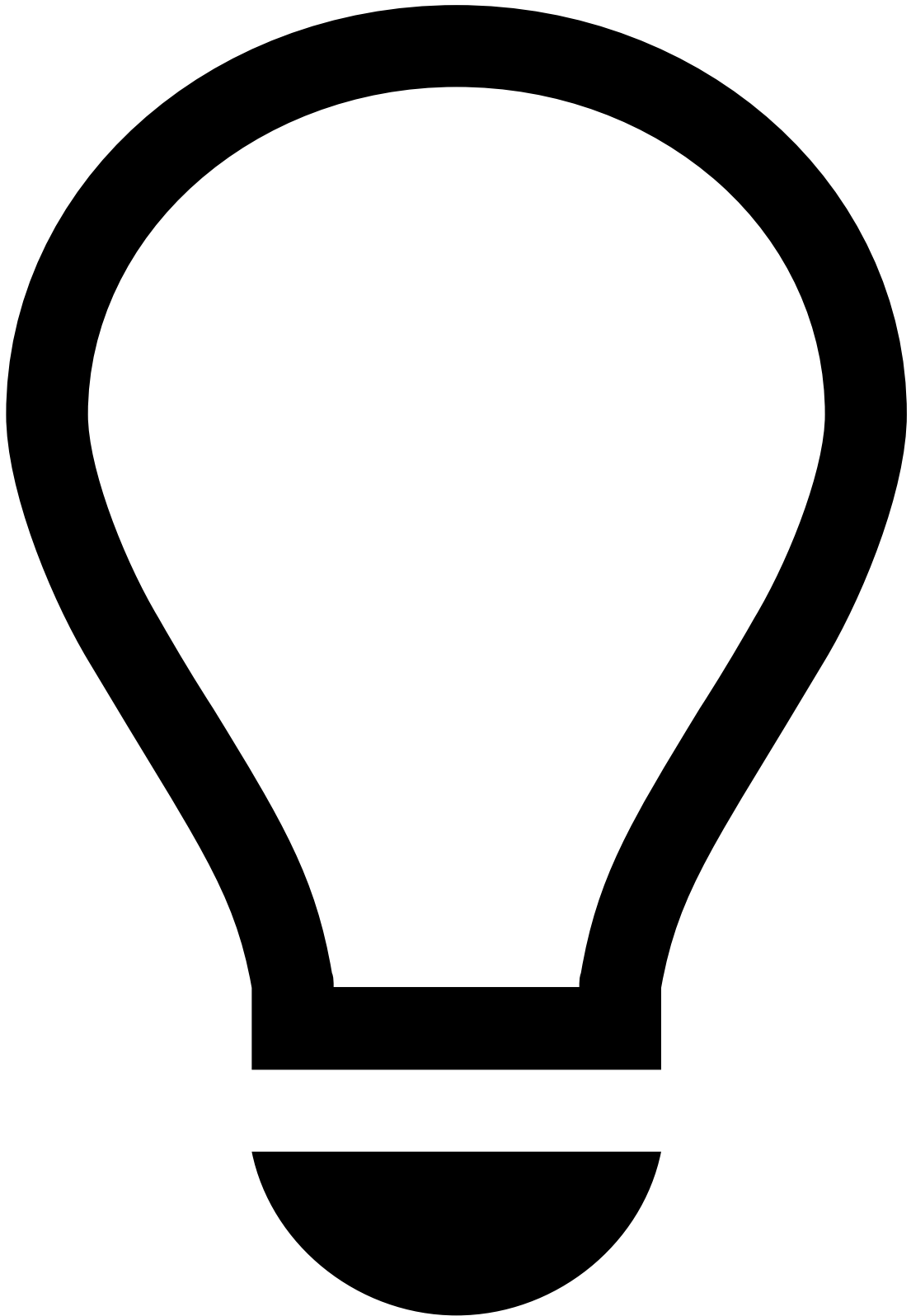
This means that even for very large datasets, with several TB in size, Pulse is very fast and efficient when computing data transformations on your [data sources](#).

Immutability

Plan executions ensure that plans, like data sources and schemas, are immutable.

When you add a deployable plan to a runtime workflow, you must specify a plan execution. The plan execution contains the plan definition at the moment it ran during the data science loop, the inputs used and the outputs created.

This isolates the plan during data science from the same plan when deployed to production. This way you can safely change the plan definition itself without affecting your production workflow.



Correcting a plan

If you need to deploy corrections to your plan, you can simply run it to create a new plan execution and generate a new snapshot of the plan. Then configure the plan service in your workflow with the new plan execution.

Data partitioning¹

A plan in Pulse is executed as a distributed job in Spark. This allows you to set up various plan operators to perform different data transformations and process large volumes of data.

Spark handles data that is organized into smaller logical chunks referred to as partitions to achieve high performance.

The way that data is re-organized into partitions is specific to the operators that you add to your plan. For example, to calculate metrics defined in the Metrics operator, Spark needs to re-organize the data by the *Group By* fields configured in this operator.

Pulse has different options available to configure data partitioning so that the data re-organization is performed as homogeneous and well distributed as possible.

Check the [Data partitioning in Metrics operator](#) page to learn more about this functionality.

Compressed file output¹

Pulse implements a configuration that enables the output of plans to be directly compressed in storage.

This feature causes the underlying files to be stored as BZip2-compressed, and thus, taking up to 8 times less space on disk. As the trade-off, plans with compressed file output are slightly slower to read when used as input for other jobs such as model training, model evaluations, among others.

Check with your Feedzai contact person how to configure plan output file compression on the server.